Estudo das Características de Qualidade de Sistemas Java

Autores: Leandro Pacheco; Lucas Maia
Disciplina: Laboratório de Experimentação de Software - Lab02
Professor: Danilo de Quadros Maia Filho

Índice

1. Introdução
2. Objetivo e Questões de Pesquisa
3. Metodologia
4. Resultados
5. Discussão dos Resultados
6. Tomadas de Decisão
7. Conclusão

1. Introdução

Este trabalho investiga as características de qualidade de sistemas desenvolvidos em Java, analisando os 1.000 repositórios Java mais populares do GitHub, com base no número de estrelas. O objetivo central é verificar se existe correlação entre atributos de processo como popularidade, maturidade, atividade e tamanho e a qualidade interna do código, medida por métricas estruturais obtidas por meio da ferramenta de análise estática CK (Chidamber & Kemerer).

A condução do estudo ocorreu em duas etapas principais. Na primeira etapa (Sprint 01), foram desenvolvidos scripts para coleta de dados via API do GitHub, além da implementação do processo de clonagem e execução da ferramenta CK em um repositório piloto. Na segunda etapa (Sprint 02), o processo foi escalado para toda a base de 1.000 repositórios, possibilitando a extração das métricas de qualidade e a realização da análise estatística.

2. Objetivo e Questões de Pesquisa

Objetivo

Investigar a relação entre métricas de processo de software e métricas de qualidade estrutural em sistemas Java, buscando identificar padrões de correlação que possam indicar fatores associados à qualidade do código.

Questões de Pesquisa

RQ 01. Qual a relação entre a popularidade dos repositórios e as suas características de qualidade?

RQ 02. Qual a relação entre a maturidade dos repositórios e as suas características de qualidade?

RQ 03. Qual a relação entre a atividade dos repositórios e as suas características de qualidade?

RQ 04. Qual a relação entre o tamanho dos repositórios e as suas características de qualidade?

As variáveis de processo consideradas foram: **número de estrelas (popularidade), idade do repositório (maturidade), quantidade de releases (atividade) e linhas de código (tamanho). As variáveis de qualidade analisadas foram: CBO, DIT e LCOM.**

Hipóteses

H1 — Popularidade implica qualidade?
Repositórios com mais estrelas tendem a ter código com menor acoplamento (CBO baixo) e maior coesão (LCOM baixo), já que a visibilidade atrai mais atenção e contribuições para correção

H2 — Maturidade implica qualidade?
Repositórios mais antigos acumulam mais refatorações ao longo dos anos, resultando em métricas de qualidade melhores.

H3 — Atividade e qualidade estão correlacionadas?
Repositórios com mais releases tendem a ter melhor qualidade, pois ciclos frequentes de entrega incentivam a manutenção contínua.

H4 — Repositórios maiores têm pior qualidade?
Conforme o código cresce em linhas de código (LOC), a complexidade estrutural tende a aumentar (CBO e LCOM mais altos), pois fica mais difícil manter a modularidade.

3. Metodologia

3.1 Seleção dos Repositórios

Foram selecionados os 1.000 repositórios Java mais populares do GitHub, utilizando como critério o número de estrelas. A coleta foi realizada por meio da API REST da plataforma.

Para cada repositório, foram extraídos os seguintes atributos: **número de estrelas, idade em anos, quantidade de releases, tamanho em kilobytes e URL de clonagem.**

3.2 Métricas de Qualidade

As métricas de qualidade foram obtidas por meio da ferramenta **CK v0.7.0**, que realiza análise estática em arquivos Java. Foram consideradas as seguintes métricas:

Métrica	Nome Completo	O que mede
CBO	Coupling Between Objects	Acoplamento entre classes
DIT	Depth of Inheritance Tree	Profundidade da herança
LCOM	Lack of Cohesion of Methods	Falta de coesão
LOC	Lines of Code	Linhas de código

Para cada repositório, foi calculada a mediana das métricas considerando todas as classes analisadas, a fim de reduzir o impacto de valores extremos.

3.3 Pipeline de Execução

O processamento dos repositórios foi realizado por meio de um pipeline automatizado composto pelas seguintes etapas:

1. Clonagem do repositório utilizando shallow clone
2. Execução da ferramenta CK
3. Extração das métricas geradas
4. Consolidação dos dados em arquivo único
5. Remoção do repositório local

Foram aplicadas otimizações como execução paralela, uso de diretórios temporários, retomada automática em caso de falhas e definição de limites de tempo para evitar travamentos.

Otimizações

Técnica	Benefício
Shallow clone	Clone ~10x mais rápido, sem histórico git
Paralelismo	3 repositórios analisados simultaneamente
Diretório temporário	Disco ocupado: ~2-5 GB (vs ~100 GB se clonasse todos antes)
Retomável	Progresso salvo incrementalmente; se interrompido, retoma de onde parou
Timeout	Clone: 180s, CK: 600s, evita que repos gigantes travem o pipeline

O tempo total de execução foi de aproximadamente 1 hora e 17 minutos.

3.4 Dados Resultantes

Dos 1.000 repositórios inicialmente selecionados, 963 foram analisados com sucesso. Dentre estes, 938 possuíam pelo menos cinco classes Java, sendo considerados válidos para a análise. Um total de 37 repositórios foi descartado por não conter código relevante ou por falhas na execução.

Item	Quantidade
Repositórios totais	1.000
Sucesso	963
Amostra válida	938
Falhas	37

3.5 Análise Estatística

Foi utilizado o coeficiente de correlação de Spearman (ρ), apropriado para análise de relações monotônicas entre variáveis. A interpretação adotada foi:

Intervalo de |ρ|	Interpretação
< 0,20	Correlação fraca
0,20 – 0,39	Correlação moderada
0,40 – 0,69	Correlação substancial
≥ 0,70	Correlação forte

4. Resultados

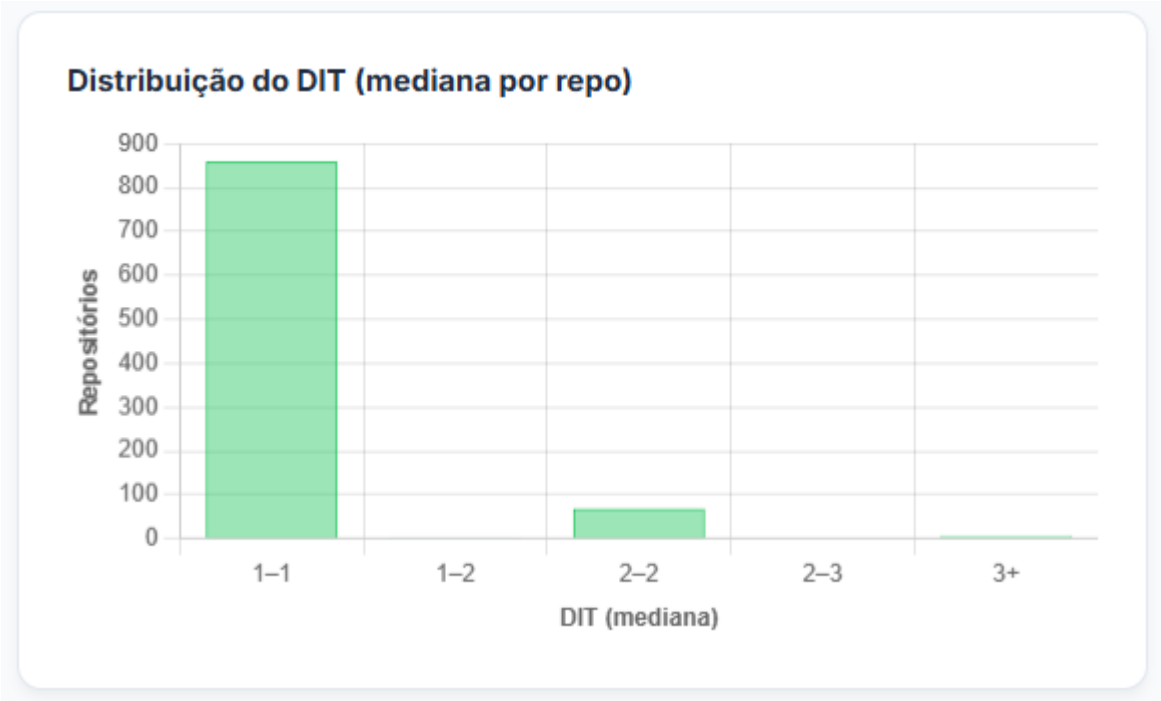
4.1 Visão Geral das Métricas

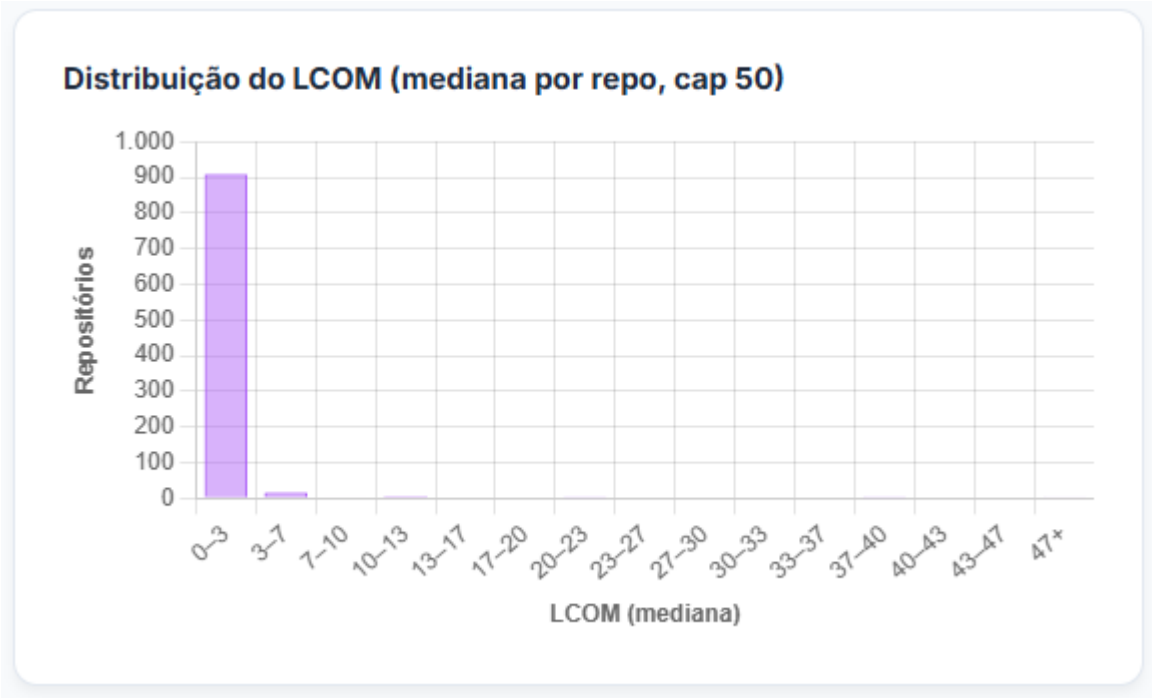
Observações iniciais:

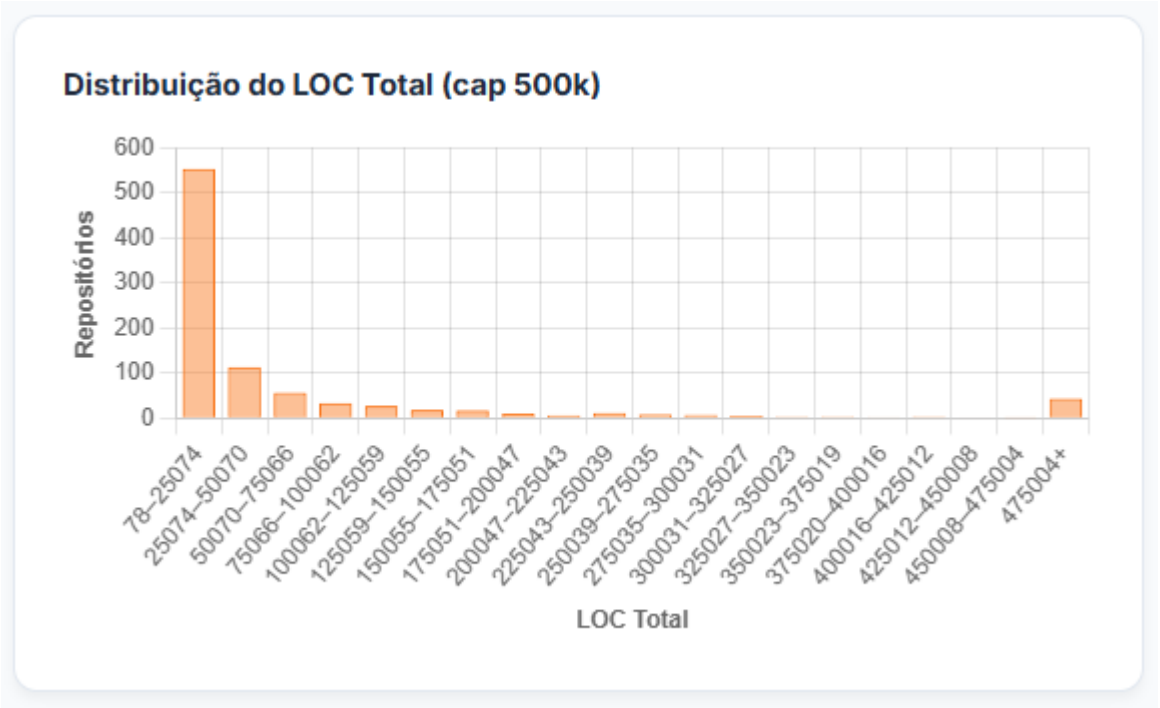
- O CBO mediano de 3.0 indica baixo acoplamento típico: cada classe referencia apenas 3 outras em mediana.
- O DIT mediano de 1.0 revela hierarquias de herança rasas, a maioria das classes herda diretamente de Object.
- O LCOM mediano de 0.0 sugere boa coesão (métodos que compartilham atributos), mas o desvio altíssimo (7.11) revela outliers extremos.
- A diferença entre mediana (15.8k) e média (89.6k) do LOC evidencia distribuição fortemente assimétrica.

Métrica	Mediana	Média	Desvio Padrão	Min	Max
CBO	3.00	3.59	1.69	0	30
DIT	1.00	1.09	0.30	1	3
LCOM	0.00	1.01	7.11	0	190
LOC	15.820	89.643	223.671	78	2.265.956
Classes	359	1.598	3.625	5	44.600

Distribuição do CBO (mediana por repo)
Repositórios
450
400
350
300
250
200
150
100
50
0
0–2
2–4
4–6
6–8
8–10
10–12
12–14
14–16
16–18
18–20
20–22
22–24
24–26
26–28
28+
CBO (mediana)


Distribuição do DIT (mediana por repo)
Repositórios
900
800
700
600
500
400
300
200
100
0
1–1
1–2
2–2
2–3
3+
DIT (mediana)


Distribuição do LCOM (mediana por repo, cap 50)
Repositórios
1.000
900
800
700
600
500
400
300
200
100
0
0–3
3–7
7–10
10–13
13–17
17–20
20–23
23–27
27–30
30–33
33–37
37–40
40–43
43–47
47+
LCOM (mediana)


Distribuição do LOC Total (cap 500k)
Repositórios
600
500
400
300
200
100
0
78–25074
25074–50070
50070–75066
75066–100062
100062–125059
125059–150055
150055–175051
175051–200047
200047–225043
225043–250039
250039–275035
275035–300031
300031–325027
325027–350023
350023–375019
375020–400016
400016–425012
425012–450008
450008–475004
475004+
LOC Total

4.2 Correlações de Spearman

A análise de correlação indicou relações predominantemente fracas a moderadas entre as variáveis de processo e as métricas de qualidade.

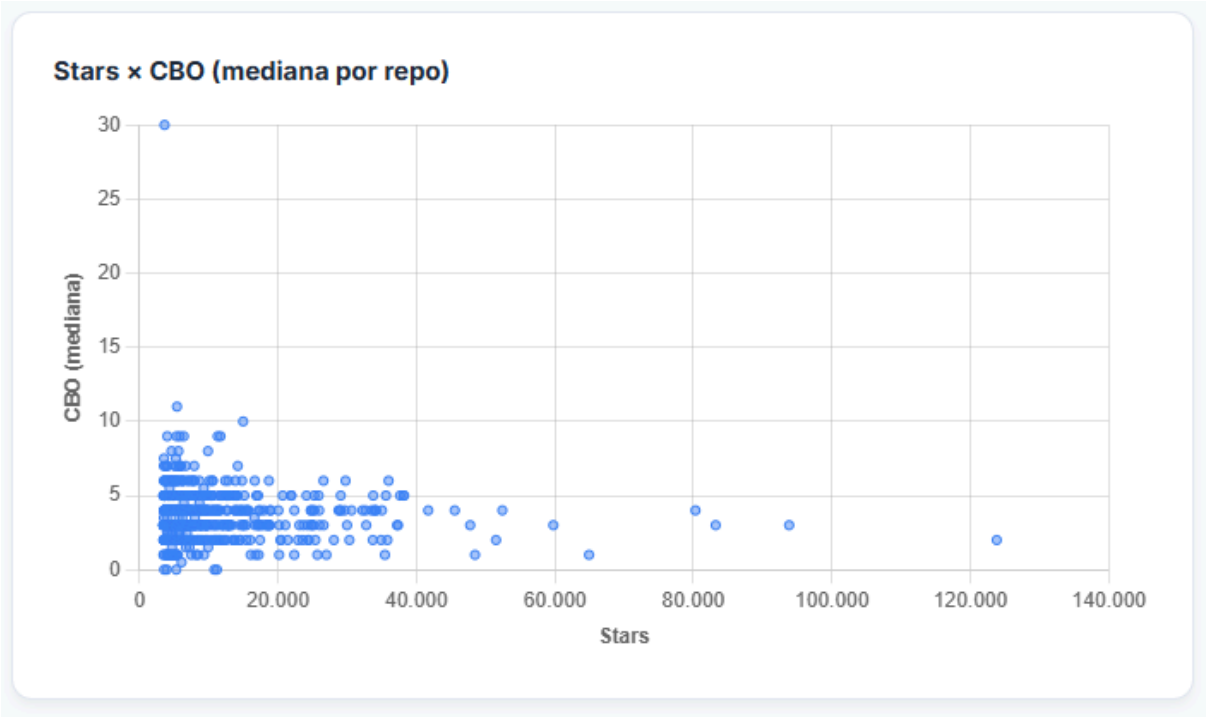
Métrica de Processo	CBO	DIT	LCOM
Popularidade (Stars)	0.08 Fraca	0.37 Moderada	0.14 Fraca
Maturidade (Idade)	0.03 Fraca	0.44 Substancial	0.19 Fraca
Atividade (Releases)	0.30 Moderada	0.37 Moderada	0.23 Moderada
Tamanho (LOC)	0.30 Moderada	0.37 Moderada	0.25 Moderada

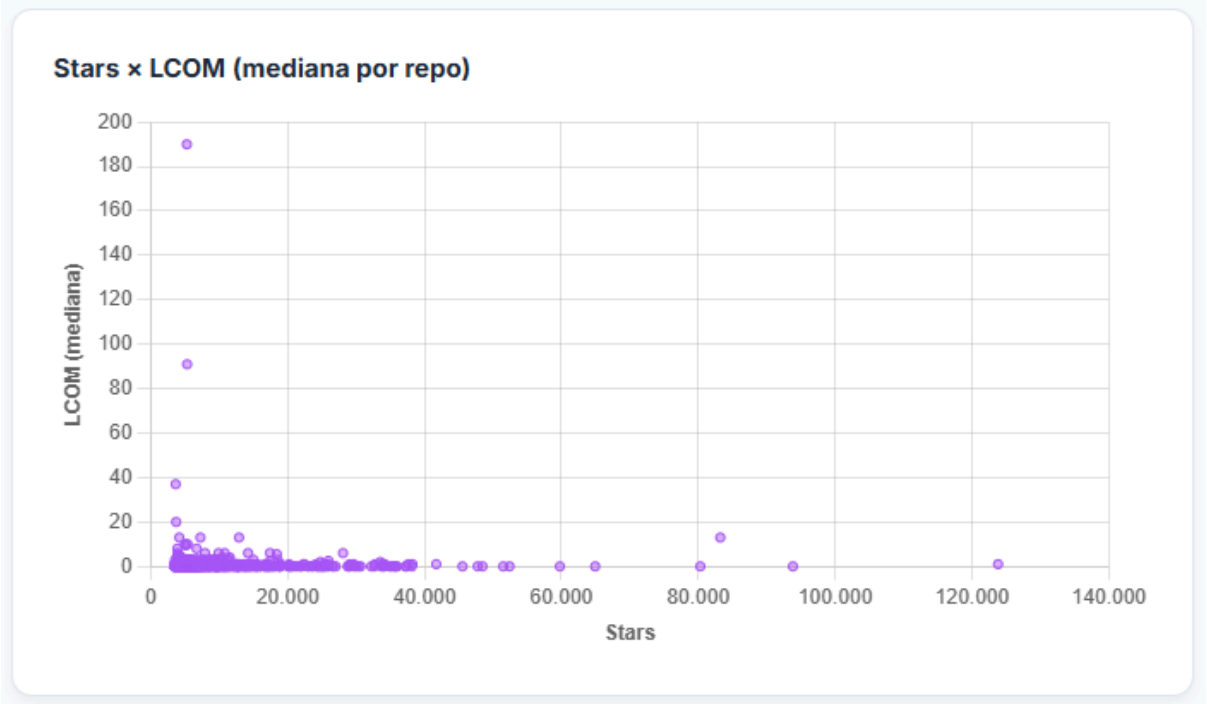
Interpretação: |ρ| < 0.20 = Fraca | 0.20–0.39 = Moderada | 0.40–0.69 = Substancial | ≥ 0.70 = Forte

4.3 Popularidade X Qualidade - RQ01

A análise evidenciou que a correlação entre o número de estrelas e as métricas de qualidade é predominantemente fraca, com valores de ρ = 0,08 para CBO e ρ = 0,14 para LCOM. Apenas a métrica DIT apresentou correlação moderada (ρ = 0,37). Esses resultados indicam que a popularidade dos repositórios não constitui um fator determinante da qualidade estrutural do código, apresentando influência limitada e inconsistente entre as diferentes métricas analisadas.

Quartil	Stars	CBO	LCOM
Q1	3.474 – 4.700	3.00	0.00
Q2	4.700 – 5.739	3.00	0.00
Q3	5.739 – 8.600	3.00	0.00
Q4	8.600 – 123.816	4.00	0.00

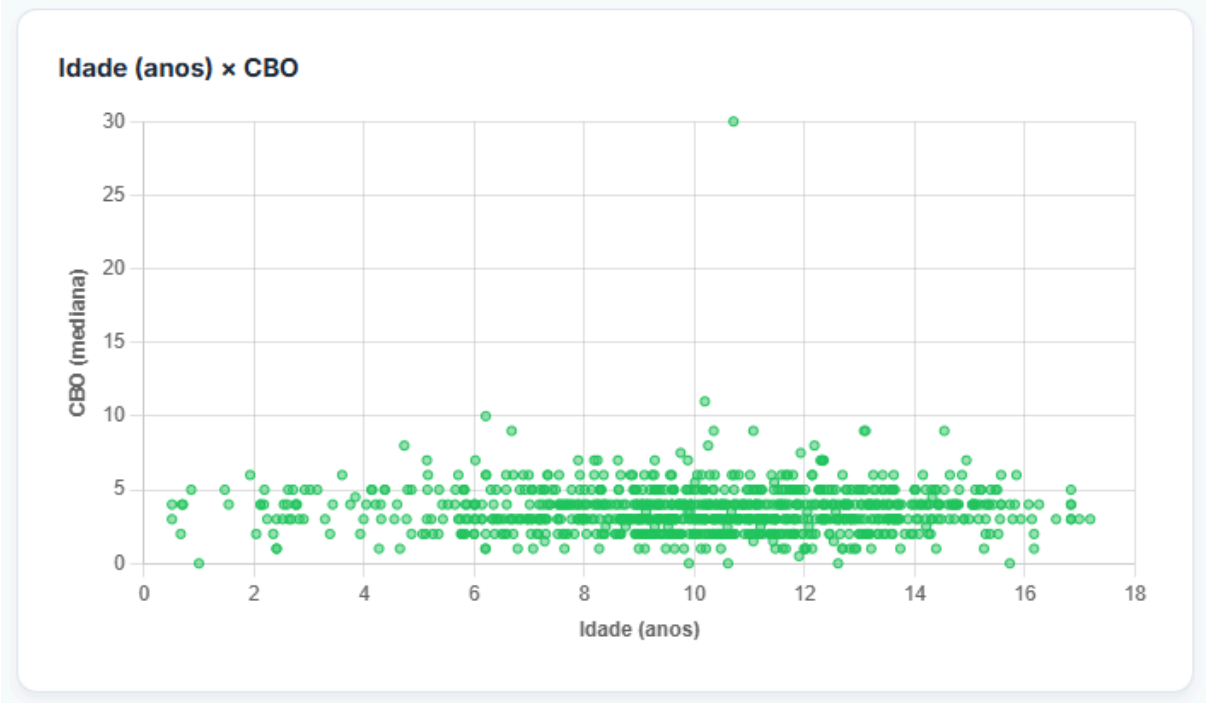

Stars × CBO (mediana por repo)
CBO (mediana)
30
25
20
15
10
5
0
0
20.000
40.000
60.000
80.000
100.000
120.000
140.000
Stars

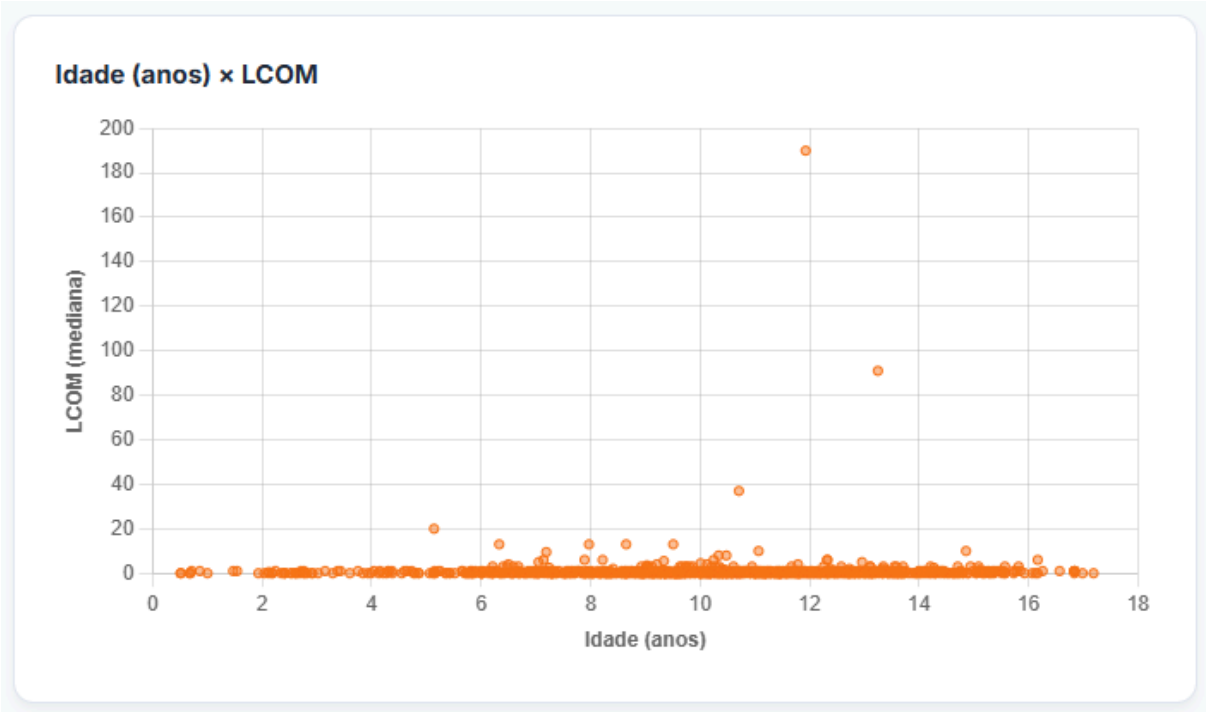

Stars × LCOM (mediana por repo)
LCOM (mediana)
200
180
160
140
120
100
80
60
40
20
0
0
20.000
40.000
60.000
80.000
100.000
120.000
140.000
Stars

A popularidade não garante qualidade. Repos extremamente populares (Q4) apresentam CBO ligeiramente maior, sugerindo que projetos maiores e mais usados tendem a ter mais dependências entre classes.

4.4 Maturidade X Qualidade - RQ02

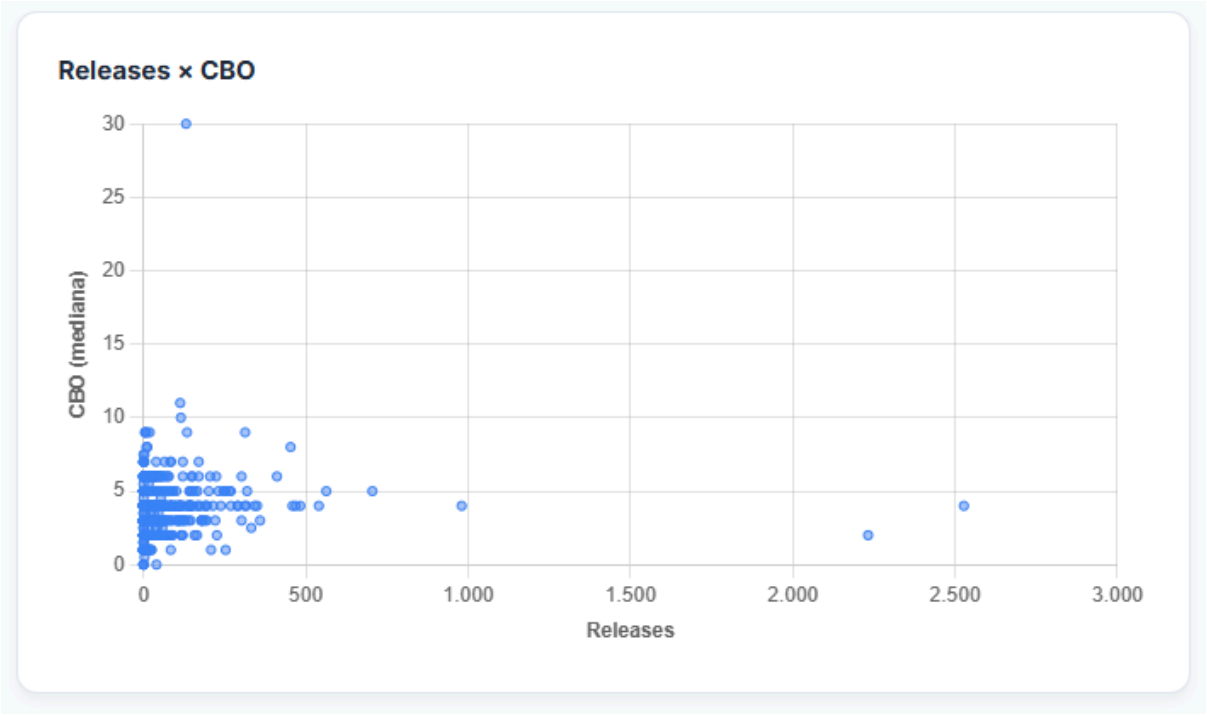
A maturidade apresentou correlação fraca com o acoplamento (ρ = 0,03 para CBO) e sem impacto relevante na coesão. Em contrapartida, foi observada correlação substancial com a profundidade da herança (ρ = 0,44 para DIT), indicando que sistemas mais antigos tendem a utilizar mais herança em sua arquitetura.

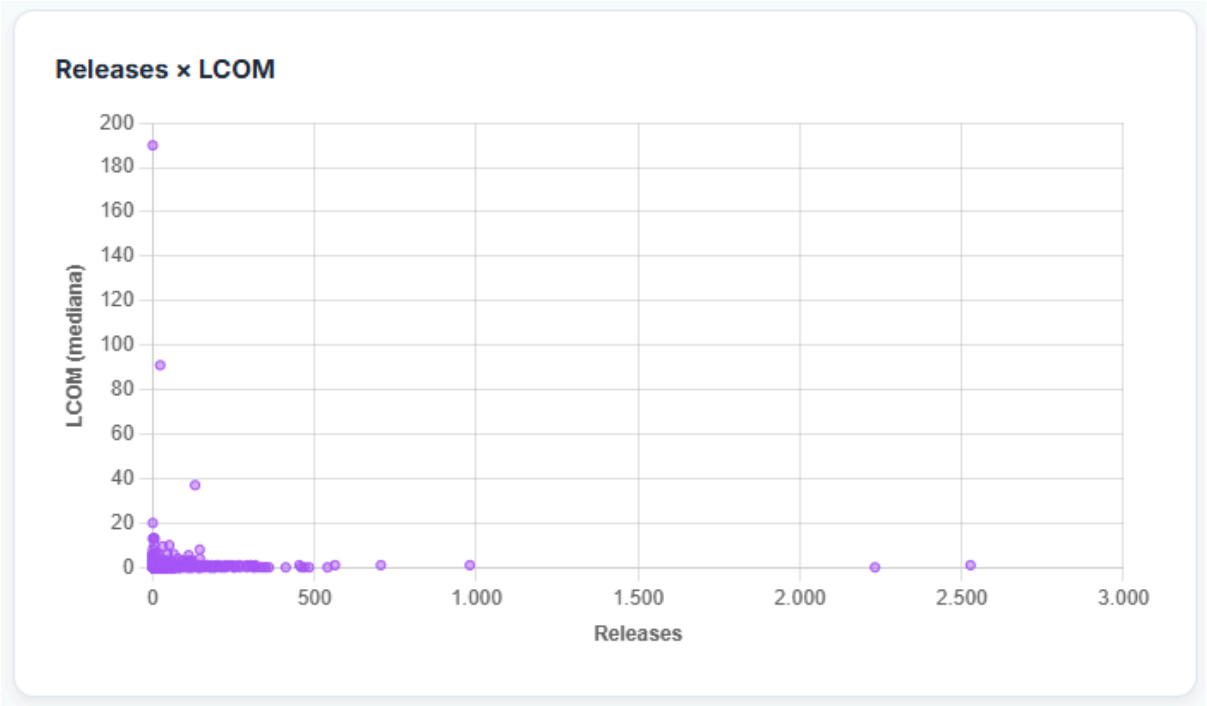




4.5 Atividade X Qualidade - RQ03

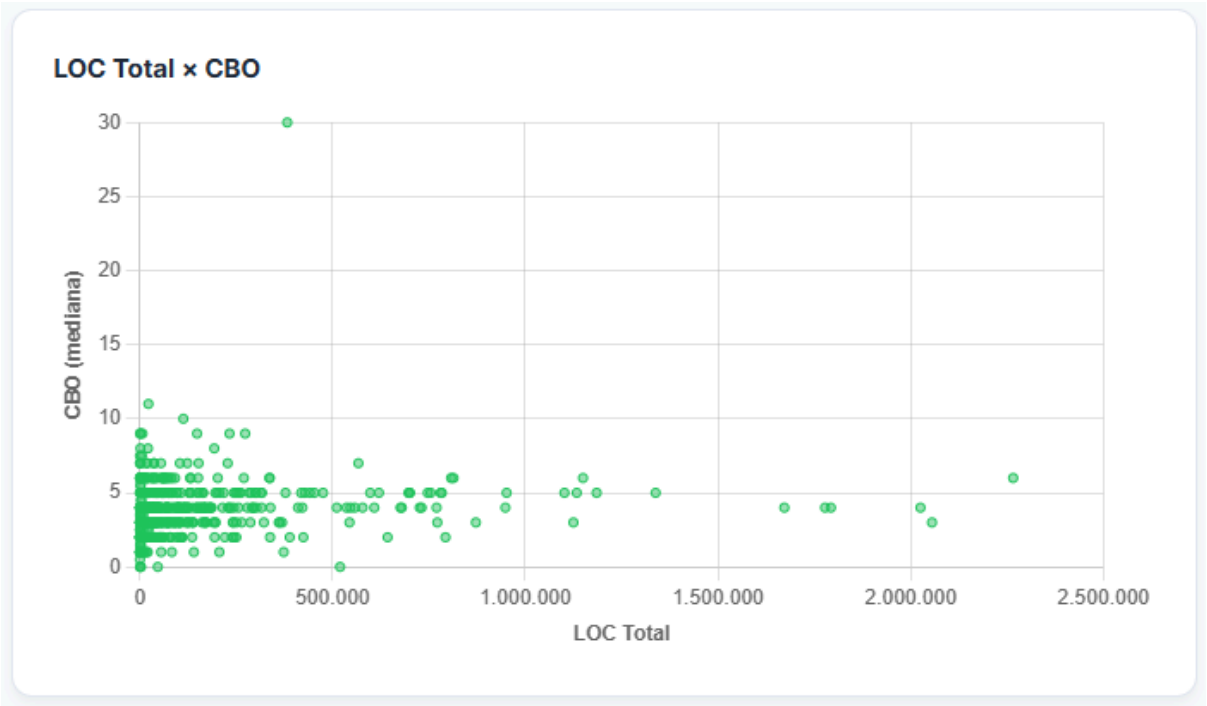
A atividade apresentou correlação moderada com as métricas estruturais, com ρ = 0,30 para CBO e ρ = 0,37 para DIT. Esses resultados indicam que repositórios com maior número de releases tendem a apresentar maior acoplamento e estruturas de herança mais complexas. Uma possível explicação é que projetos com ciclos frequentes de entrega geralmente correspondem a sistemas mais maduros e funcionalmente ricos, nos quais o aumento da complexidade estrutural ocorre como consequência natural da evolução contínua e da ampliação de funcionalidades.

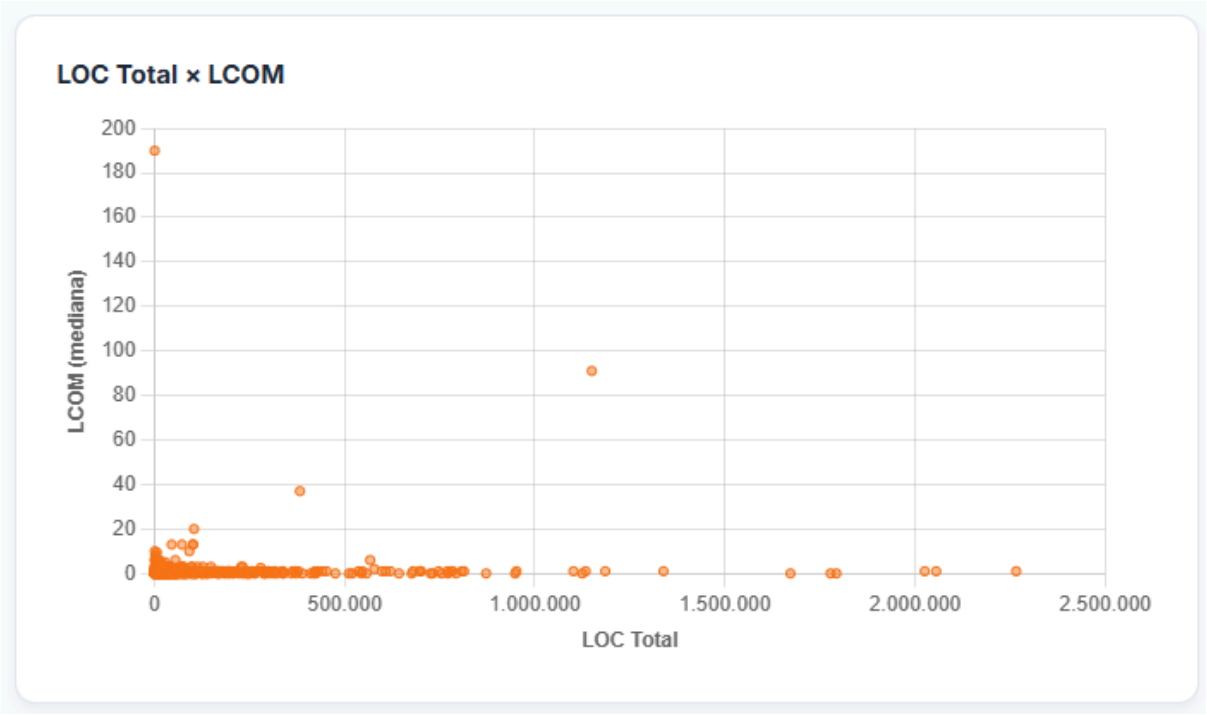




4.6 Tamanho X Qualidade - RQ04

O tamanho do sistema apresentou correlação moderada com as métricas de qualidade, com ρ = 0,30 para CBO e ρ = 0,25 para LCOM. Esses resultados indicam que projetos maiores tendem a apresentar maior acoplamento e menor coesão. Esse comportamento é esperado, uma vez que, conforme o sistema cresce em linhas de código, torna-se mais difícil manter níveis ideais de modularidade, levando ao acúmulo de interdependências entre classes.





5. Discussão dos Resultados

A análise dos resultados foi conduzida com base nas hipóteses previamente estabelecidas, permitindo uma avaliação direta das relações entre as métricas de processo e as características de qualidade dos repositórios.

H1 — Popularidade implica qualidade?
Refutada. A correlação entre o número de estrelas e as métricas de qualidade mostrou-se fraca ($\rho < 0{,}15$ para CBO e LCOM). Isso indica que a popularidade no GitHub está mais relacionada a fatores como utilidade prática, qualidade da documentação e engajamento da comunidade, não refletindo diretamente a qualidade estrutural do código. Observou-se ainda que repositórios extremamente populares apresentaram valores de CBO ligeiramente maiores, sugerindo aumento de dependências em sistemas mais amplamente utilizados.

H2 — Maturidade implica qualidade?
Refutada parcialmente. A idade dos repositórios não apresentou correlação significativa com acoplamento (CBO) ou coesão (LCOM), porém mostrou correlação substancial com a profundidade de herança (DIT), com $\rho = 0{,}44$. Esse resultado sugere que sistemas mais antigos tendem a adotar estruturas mais baseadas em herança, refletindo padrões arquiteturais predominantes em versões anteriores do Java.

H3 — Atividade correlaciona com qualidade?
Confirmada parcialmente. Foi observada correlação moderada ($\rho \approx 0{,}30$) entre o número de releases e as métricas CBO e DIT. Entretanto, essa relação ocorre de forma inversa ao esperado: sistemas com maior atividade apresentam maior acoplamento. Isso pode ser explicado pelo fato de que projetos com maior frequência de releases tendem a ser sistemas mais complexos e funcionalmente ricos, nos quais o aumento da complexidade estrutural é uma consequência natural da evolução contínua.

H4 — Repositórios maiores têm pior qualidade?
Confirmada parcialmente. A análise indicou correlação moderada entre o tamanho do código (LOC) e as métricas de qualidade ($\rho = 0{,}30$ para CBO e 0,25 para LCOM). Esse resultado confirma que, à medida que os sistemas crescem, torna-se mais difícil manter níveis ideais de modularidade e coesão. No entanto, o efeito observado é moderado, indicando que, apesar do crescimento, os mantenedores ainda conseguem controlar parcialmente a complexidade estrutural.

6. Tomadas de Decisão

Com base nos resultados obtidos, foram definidas as seguintes decisões:

- Rejeitar a hipótese de que popularidade implica qualidade
- Considerar a maturidade relevante apenas para aspectos arquiteturais
- Interpretar atividade como indicador de complexidade e não de qualidade
- Utilizar tamanho do sistema como principal variável explicativa
- Adotar paralelização do processamento com múltiplos workers, permitindo a análise simultânea de repositórios e reduzindo o tempo total de execução
- Limitar a execução paralela a 3 workers, garantindo equilíbrio entre desempenho e estabilidade do ambiente
- Implementar a remoção automática dos repositórios após análise, evitando acúmulo excessivo de dados em disco
- Utilizar estratégias de otimização de armazenamento como prática obrigatória em execuções em larga escala

7. Conclusão

A análise realizada com 938 repositórios Java demonstrou que a qualidade estrutural do código apresenta correlação predominantemente fraca a moderada com métricas de processo.

Os principais achados indicam que popularidade não é sinônimo de qualidade, maturidade influencia padrões arquiteturais, e o tamanho do sistema é o fator mais associado ao aumento da complexidade estrutural.

Apesar disso, os resultados mostram que, em geral, os repositórios analisados apresentam níveis satisfatórios de qualidade, com baixo acoplamento e boa coesão.

Limitações

Este estudo apresenta algumas limitações, incluindo a restrição a repositórios populares, falhas na análise de alguns projetos, limitações da ferramenta utilizada e ausência de análise histórica da evolução do código.